09 — Evaluación y Comparación de Modelos

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook evalúa los cinco modelos con las **predicciones out-of-fold (OOF)** de la validación cruzada del notebook 08 sobre las **8274 imágenes**. No hay hold-out ni test separado: la estimación es la CV OOF.

Métricas

- Accuracy, balanced accuracy, precision, recall y F1 (macro y weighted).
- Métricas por clase (`health` , `sick` , `tb`).
- AUC-ROC por clase (One-vs-Rest) y **Average Precision (AP)** de `tb` .
- Intervalos de confianza por bootstrap y McNemar con Holm.
- Calibración de `tb` (Brier + curva).

Protocolo

1. Cargar las predicciones y scores OOF del notebook 08.
2. Evaluar las 8274 imágenes (cada una predicha exactamente una vez).
3. Comparar modelos y analizar la incertidumbre.
4. Recomendar el modelo elegido por la CV del 08.

Flujo

```
data/08_oof_predicciones.csv
data/08_oof_scores.npz
data/08_manifest.json
        |
        v
 Metricas globales + por clase + AUC/AP
        |
        v
 Matrices de confusion + curvas ROC/AP
        |
        v
 Bootstrap + McNemar (global y tb) + calibracion
        |
        v
 Recomendacion final
```

0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las librerías de evaluación.
# Referencias:
#   metricas: https://scikit-learn.org/stable/modules/model_evaluation.html
#   calibracion: https://scikit-learn.org/stable/modules/calibration.html

import sys, os, warnings, json
warnings.filterwarnings('ignore')

from pathlib import Path
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.stats import chi2

from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay,
    confusion_matrix,
    roc_curve,
    auc,
    precision_recall_curve,
    average_precision_score,
    roc_auc_score,
    brier_score_loss,
)
from sklearn.calibration import calibration_curve

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

CLASS_NAMES = ['health', 'sick', 'tb']
N_BOOTSTRAP = 1000

DATA_DIR    = Path('..') / 'data'
REPORTS_DIR = Path('..') / 'reports'
REPORTS_DIR.mkdir(exist_ok=True)

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga de las predicciones out-of-fold

In [2]:

```
# Se cargan las predicciones y scores OOF del notebook 08.
# Cada imagen fue predicha exactamente una vez por el modelo de su fold externo.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.load.html

for archivo in ['08_oof_predicciones.csv', '08_oof_scores.npz', '08_manifest.json']:
    if not (DATA_DIR / archivo).exists():
        raise FileNotFoundError(
            f'No se encontró {archivo}. Ejecuta primero el notebook 08.'
        )

oof = pd.read_csv(DATA_DIR / '08_oof_predicciones.csv')
score_data = np.load(DATA_DIR / '08_oof_scores.npz')

y_true = oof['y_true'].to_numpy()
MODELOS = [c for c in oof.columns if c not in ('y_true', 'Dummy')]
predicciones = {n: oof[n].to_numpy() for n in MODELOS}
puntuaciones = {n: score_data[n] for n in MODELOS}

with open(DATA_DIR / '08_manifest.json', encoding='utf-8') as f:
    manifest = json.load(f)

print(f'Imágenes OOF : {len(y_true)}')
print('Distribución:')
for clase, conteo in zip(*np.unique(y_true, return_counts=True)):
    print(f'  {CLASS_NAMES[clase]:6s} (label={clase}): {conteo} muestras')
print()
```

```
print('Modelos:', MODELOS)
print(f'Modelo seleccionado por CV: {manifest.get("seleccionado_cv")}')
print(f'Estrategia : {manifest.get("estrategia")}')
print(f'Selector   : {manifest.get("selector")}')
```

```
Imágenes OOF : 8256
Distribución:
  health (label=0): 3800 muestras
  sick   (label=1): 3656 muestras
  tb     (label=2): 800 muestras

Modelos: ['Logistic Regression', 'SVM', 'Decision Tree', 'Random Forest', 'XGBoost']
Modelo seleccionado por CV: XGBoost
Estrategia : class_weight
Selector   : {'metodo': 'MI', 'n_features': 2000}
```

2. Comparación global

In [3]:

```
# Métricas globales sobre las 8274 predicciones OOF.
# AP = Average Precision (average_precision_score); no es el área trapezoidal de la PR.

filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    y_score = puntuaciones[nombre]
    filas.append({
        'Modelo': nombre,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Balanced Acc': balanced_accuracy_score(y_true, y_pred),
        'Precision macro': precision_score(y_true, y_pred, average='macro', zero_division=0),
        'Recall macro': recall_score(y_true, y_pred, average='macro', zero_division=0),
        'F1 macro': f1_score(y_true, y_pred, average='macro', zero_division=0),
        'Precision weighted': precision_score(y_true, y_pred, average='weighted', zero_division=0),
        'Recall weighted': recall_score(y_true, y_pred, average='weighted', zero_division=0),
        'F1 weighted': f1_score(y_true, y_pred, average='weighted', zero_division=0),
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None, zero_division=0)[0],
        'AUC-ROC macro': np.mean([
            roc_auc_score((y_true == c).astype(int), y_score[:, c])
            for c in range(len(CLASS_NAMES))
        ]),
        'AP tb': average_precision_score((y_true == 2).astype(int), y_score[:, 2]),
    })

resultados_globales = pd.DataFrame(filas).sort_values(
    ['Balanced Acc', 'F1 macro'], ascending=False,
).reset_index(drop=True)

print('Comparación global out-of-fold (8274 imágenes):')
print(resultados_globales.round(4).to_string(index=False))

resultados_globales.to_csv(DATA_DIR / '09_resultados_globales.csv', index=False)
print()
print(f'Guardado en: {(DATA_DIR / "09_resultados_globales.csv").resolve()}')
```

```
Comparación global out-of-fold (8274 imágenes):
             Modelo  Accuracy  Balanced Acc  Precision macro  Recall macro  F1 macro  Precision weighted
Recall weighted  F1 weighted  Recall tb  AUC-ROC macro  AP tb
            XGBoost    0.8465        0.7730           0.7490        0.7730    0.7578              0.8598
0.8465       0.8519     0.5675         0.9361 0.4967
Logistic Regression    0.8395        0.7714           0.7405        0.7714    0.7497              0.8620
0.8395       0.8484     0.5812         0.9316 0.4843
                SVM    0.8358        0.7652           0.7359        0.7652    0.7441              0.8585
0.8358       0.8443     0.5688         0.9083 0.3758
      Random Forest    0.8451        0.7080           0.7516        0.7080    0.7215              0.8323
0.8451       0.8360     0.3225         0.9227 0.4154
      Decision Tree    0.7009        0.6389           0.6247        0.6389    0.6137              0.7694
0.7009       0.7269     0.4662         0.8045 0.1940

Guardado en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_resultados_globales.csv
```

3. Métricas por clase

In [4]:
```
# Precision, recall y F1 por clase, más AUC-ROC One-vs-Rest.

filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    y_score = puntuaciones[nombre]
    for k, clase in enumerate(CLASS_NAMES):
        filas.append({
            'Modelo': nombre,
            'Clase': clase,
            'Precision': precision_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'Recall': recall_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'F1': f1_score(y_true, y_pred, labels=[k], average=None, zero_division=0)[0],
            'AUC-ROC (OvR)': roc_auc_score((y_true == k).astype(int), y_score[:, k]),
        })

resultados_clase = pd.DataFrame(filas)
print('Métricas por clase:')
print(resultados_clase.round(4).to_string(index=False))
resultados_clase.to_csv(DATA_DIR / '09_resultados_por_clase.csv', index=False)
print()
print(f'Guardado en: {(DATA_DIR / "09_resultados_por_clase.csv").resolve()}')
```

```
Métricas por clase:
             Modelo  Clase  Precision  Recall     F1  AUC-ROC (OvR)
Logistic Regression health     0.9232  0.9074 0.9152         0.9778
Logistic Regression   sick     0.8998  0.8255 0.8611         0.9575
Logistic Regression     tb     0.3985  0.5812 0.4728         0.8595
                SVM health     0.9125  0.9171 0.9148         0.9782
                SVM   sick     0.9046  0.8096 0.8545         0.9540
                SVM     tb     0.3906  0.5688 0.4631         0.7927
      Decision Tree health     0.8265  0.7734 0.7991         0.8823
      Decision Tree   sick     0.8311  0.6770 0.7462         0.8628
      Decision Tree     tb     0.2166  0.4662 0.2958         0.6684
      Random Forest health     0.8735  0.9197 0.8960         0.9675
      Random Forest   sick     0.8568  0.8818 0.8691         0.9518
      Random Forest     tb     0.5244  0.3225 0.3994         0.8487
            XGBoost health     0.9132  0.9166 0.9149         0.9773
            XGBoost   sick     0.8969  0.8348 0.8647         0.9581
            XGBoost     tb     0.4370  0.5675 0.4937         0.8731

Guardado en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_resultados_por_clase.csv
```

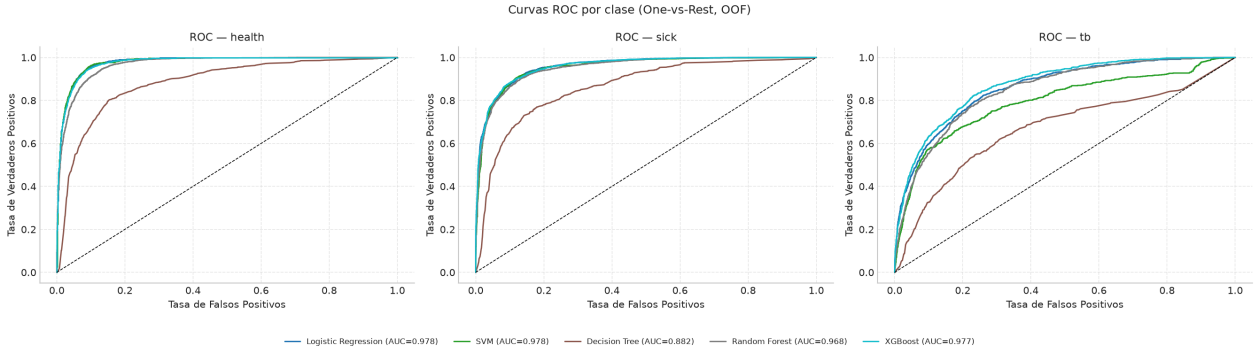
4. AUC-ROC por clase

In [5]:
```
# Curvas ROC One-vs-Rest para las 8274 predicciones OOF.

colores = plt.cm.tab10(np.linspace(0, 1, len(MODELOS)))
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (k, clase) in zip(axes, enumerate(CLASS_NAMES)):
    for color, nombre in zip(colores, MODELOS):
        y_bin = (y_true == k).astype(int)
        fpr, tpr, _ = roc_curve(y_bin, puntuaciones[nombre][:, k])
        ax.plot(fpr, tpr, color=color, lw=1.4,
                label=f'{nombre} (AUC={auc(fpr, tpr):.3f})')
    ax.plot([0, 1], [0, 1], 'k--', lw=0.8)
    ax.set_xlabel('Tasa de Falsos Positivos', fontsize=10)
    ax.set_ylabel('Tasa de Verdaderos Positivos', fontsize=10)
    ax.set_title(f'ROC — {clase}', fontsize=11)
    ax.grid(True, linestyle='--', alpha=0.4)

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower center', ncol=5, fontsize=8, frameon=False)
fig.suptitle('Curvas ROC por clase (One-vs-Rest, OOF)', fontsize=12)
fig.tight_layout(rect=[0, 0.08, 1, 1])
fig.savefig(REPORTS_DIR / '09_roc_por_clase.png', dpi=300, bbox_inches='tight')
plt.show()
```



5. Curvas Average Precision (tb)

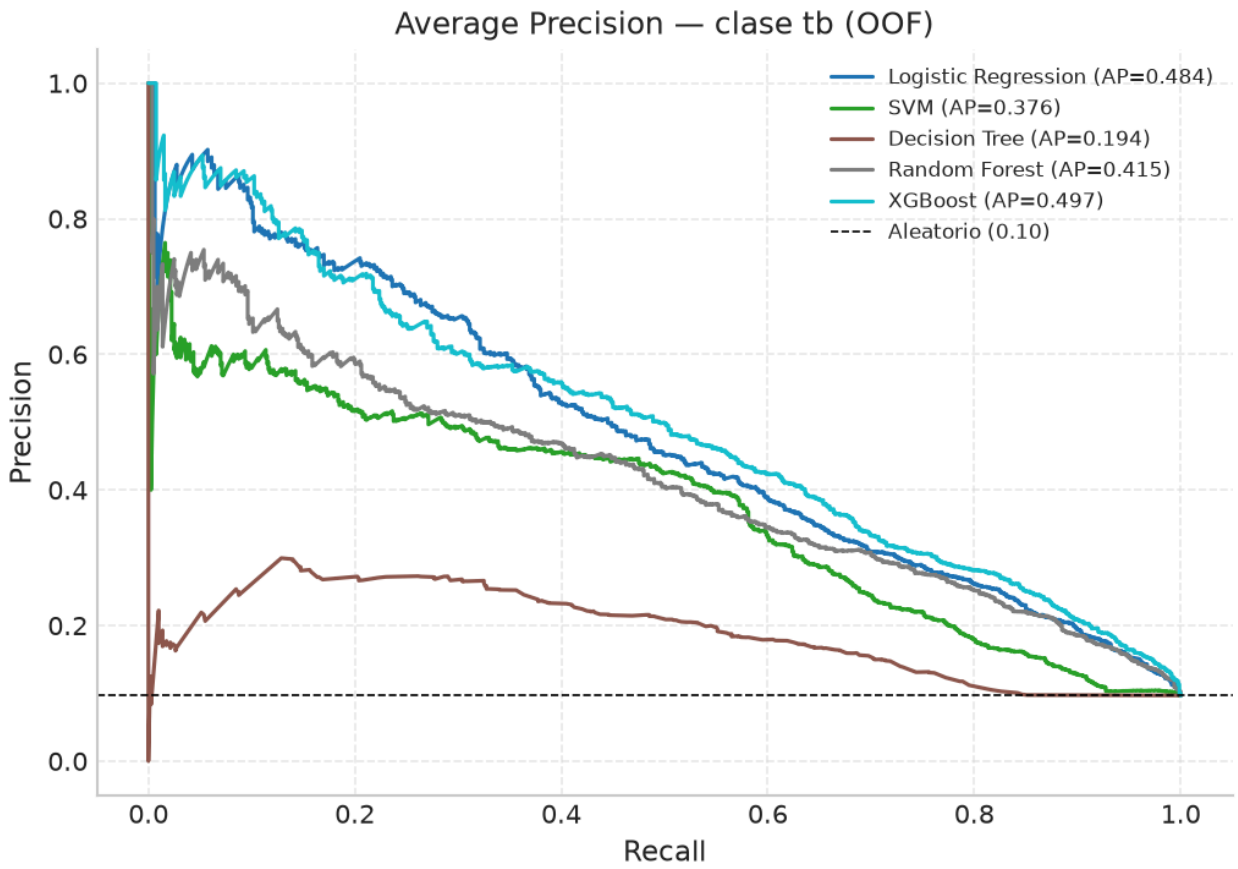
In [6]:

```
# AP (average precision) es más informativa que ROC-AUC con clase positiva minoritaria.
# Es AP, no el área trapezoidal de la curva PR.
# Referencia: https://scikit-
learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

fig, ax = plt.subplots(figsize=(7, 5))
y_tb = (y_true == 2).astype(int)

for color, nombre in zip(colores, MODELOS):
    precision, recall, _ = precision_recall_curve(y_tb, puntuaciones[nombre][:, 2])
    ap = average_precision_score(y_tb, puntuaciones[nombre][:, 2])
    ax.plot(recall, precision, color=color, lw=1.5,
            label=f'{nombre} (AP={ap:.3f})')

ax.axhline(y_tb.mean(), color='k', linestyle='--', lw=0.8,
           label=f'Aleatorio ({y_tb.mean():.2f})')
ax.set_xlabel('Recall', fontsize=11)
ax.set_ylabel('Precision', fontsize=11)
ax.set_title('Average Precision — clase tb (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_ap_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```



6. Matrices de confusión y errores

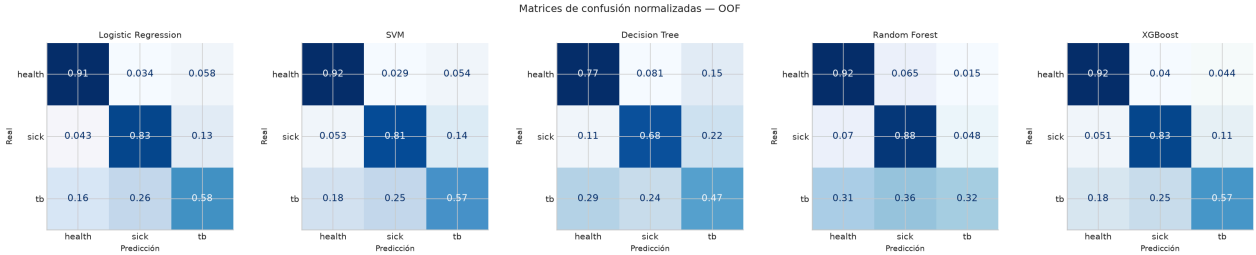
In [7]:

```python
# Matrices normalizadas por fila (recall por clase) y conteos de confusiones tb/sick.

fig, axes = plt.subplots(1, len(MODELOS), figsize=(4.2 * len(MODELOS), 4))
for ax, nombre in zip(axes, MODELOS):
    ConfusionMatrixDisplay.from_predictions(
        y_true, predicciones[nombre],
        display_labels=CLASS_NAMES,
        normalize='true', colorbar=False, cmap='Blues', ax=ax,
    )
    ax.set_title(nombre, fontsize=10)
    ax.set_xlabel('Predicción', fontsize=9)
    ax.set_ylabel('Real', fontsize=9)

fig.suptitle('Matrices de confusión normalizadas — OOF', fontsize=12, y=1.02)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_matrices_confusion.png', dpi=300, bbox_inches='tight')
plt.show()

print('Confusiones principales (conteos absolutos):')
print(f'{"Modelo":22s}  tb->sick  sick->tb  tb->health  health->tb')
for nombre in MODELOS:
    cm = confusion_matrix(y_true, predicciones[nombre])
    print(f'{nombre:22s}  {cm[2, 1]:7d}  {cm[1, 2]:7d}  {cm[2, 0]:10d}  {cm[0, 2]:9d}')
```



```
Confusiones principales (conteos absolutos):
Modelo                 tb->sick  sick->tb  tb->health  health->tb
Logistic Regression         205       481         130         221
SVM                         203       504         142         206
Decision Tree               195       796         232         553
Random Forest               291       177         251          57
XGBoost                     200       419         146         166
```

7. Comparación estadística

> Los p-valores de McNemar se corrigen con **Holm**. Se reportan el test global y uno restringido a los casos de `tb`.

In [8]:

```python
# Bootstrap estratificado para balanced accuracy y recall de tb; McNemar global y de tb.

def bootstrap_ci(y_true, y_pred, metric_fn, n_boot=N_BOOTSTRAP, seed=RANDOM_STATE):
    rng = np.random.RandomState(seed)
    clases = np.unique(y_true)
    idx = {c: np.where(y_true == c)[0] for c in clases}
    valores = []
    for _ in range(n_boot):
        muestra = np.concatenate([
            rng.choice(idx[c], size=len(idx[c]), replace=True) for c in clases
        ])
        valores.append(metric_fn(y_true[muestra], y_pred[muestra]))
    return np.percentile(valores, [2.5, 97.5])


filas = []
for nombre in MODELOS:
    y_pred = predicciones[nombre]
    bacc_lo, bacc_hi = bootstrap_ci(y_true, y_pred, balanced_accuracy_score)
    rec_lo, rec_hi = bootstrap_ci(
        y_true, y_pred,
        lambda yt, yp: recall_score(yt, yp, labels=[2], average=None)[0],
    )
    filas.append({
        'Modelo': nombre,
        'BalAcc': balanced_accuracy_score(y_true, y_pred),
        'BalAcc IC95 inf': bacc_lo,
        'BalAcc IC95 sup': bacc_hi,
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None)[0],
        'Recall tb IC95 inf': rec_lo,
        'Recall tb IC95 sup': rec_hi,
    })

intervalos = pd.DataFrame(filas).sort_values('BalAcc', ascending=False)
print('Intervalos de confianza al 95 % (bootstrap estratificado):')
print(intervalos.round(4).to_string(index=False))


def mcnemar(y_true, pred_a, pred_b):
    a_ok = pred_a == y_true
    b_ok = pred_b == y_true
    n01 = np.sum(a_ok & ~b_ok)
    n10 = np.sum(~a_ok & b_ok)
    if n01 + n10 == 0:
        return np.nan, np.nan
    stat = (abs(n01 - n10) - 1) ** 2 / (n01 + n10)
    return stat, chi2.sf(stat, 1)


def tabla_mcnemar(y_eval, preds):
    nombres = list(preds)
    pares = []
    for i in range(len(nombres)):
        for j in range(i + 1, len(nombres)):
            _, p = mcnemar(y_eval, preds[nombres[i]], preds[nombres[j]])
            if p != p:
                p = 1.0
```

```
            pares.append({'Modelo A': nombres[i], 'Modelo B': nombres[j], 'p': p})
    m = len(pares)
    orden = sorted(range(m), key=lambda k: pares[k]['p'])
    anterior = 0.0
    for rango, k in enumerate(orden):
        ajustado = min(1.0, (m - rango) * pares[k]['p'])
        ajustado = max(ajustado, anterior)
        anterior = ajustado
        pares[k]['p Holm'] = ajustado
    return pares


pares = tabla_mcnemar(y_true, predicciones)
print()
print('McNemar global (OOF; p crudo y Holm):')
print(f'{"Modelo A":22s} {"Modelo B":22s} {"p":>8s} {"p Holm":>8s} {"significativo":>14s}')
for par in sorted(pares, key=lambda d: d['p']):
    sig = 'sí' if par['p Holm'] < 0.05 else 'no'
    print(f'{par["Modelo A"]:22s} {par["Modelo B"]:22s} '
          f'{par["p"]:8.4f} {par["p Holm"]:8.4f} {sig:>14s}')

mask_tb = y_true == 2
pares_tb = tabla_mcnemar(y_true[mask_tb], {n: p[mask_tb] for n, p in predicciones.items()})
print()
print('McNemar restringido a tb (OOF; p crudo y Holm):')
print(f'{"Modelo A":22s} {"Modelo B":22s} {"p":>8s} {"p Holm":>8s} {"significativo":>14s}')
for par in sorted(pares_tb, key=lambda d: d['p']):
    sig = 'sí' if par['p Holm'] < 0.05 else 'no'
    print(f'{par["Modelo A"]:22s} {par["Modelo B"]:22s} '
          f'{par["p"]:8.4f} {par["p Holm"]:8.4f} {sig:>14s}')
```

```
Intervalos de confianza al 95 % (bootstrap estratificado):
             Modelo  BalAcc  BalAcc IC95 inf  BalAcc IC95 sup  Recall tb  Recall tb IC95 inf  Recall tb
IC95 sup
            XGBoost  0.7730           0.7611           0.7851     0.5675              0.5350
0.6012
Logistic Regression  0.7714           0.7592           0.7835     0.5812              0.5500
0.6150
                SVM  0.7652           0.7525           0.7776     0.5688              0.5337
0.6025
      Random Forest  0.7080           0.6959           0.7203     0.3225              0.2900
0.3563
      Decision Tree  0.6389           0.6262           0.6524     0.4662              0.4312
0.5000

McNemar global (OOF; p crudo y Holm):
Modelo A               Modelo B                     p   p Holm  significativo
Decision Tree          Random Forest           0.0000   0.0000             sí
Decision Tree          XGBoost                 0.0000   0.0000             sí
Logistic Regression    Decision Tree           0.0000   0.0000             sí
SVM                    Decision Tree           0.0000   0.0000             sí
SVM                    XGBoost                 0.0033   0.0196             sí
SVM                    Random Forest           0.0231   0.1154             no
Logistic Regression    XGBoost                 0.0473   0.1893             no
Logistic Regression    SVM                     0.1456   0.4368             no
Logistic Regression    Random Forest           0.1641   0.4368             no
Random Forest          XGBoost                 0.6871   0.6871             no

McNemar restringido a tb (OOF; p crudo y Holm):
Modelo A               Modelo B                     p   p Holm  significativo
Random Forest          XGBoost                 0.0000   0.0000             sí
Logistic Regression    Random Forest           0.0000   0.0000             sí
SVM                    Random Forest           0.0000   0.0000             sí
Decision Tree          Random Forest           0.0000   0.0000             sí
Logistic Regression    Decision Tree           0.0000   0.0000             sí
Decision Tree          XGBoost                 0.0000   0.0000             sí
SVM                    Decision Tree           0.0000   0.0000             sí
Logistic Regression    SVM                     0.2888   0.8665             no
Logistic Regression    XGBoost                 0.4418   0.8835             no
SVM                    XGBoost                 1.0000   1.0000             no
```

8. Calibración de probabilidades (tb)

In [9]:
```python
# Brier score y curva de calibración para tb sobre los scores OOF.
# Solo modelos con predict_proba: LR, DT, RF y XGBoost. SVM no expone probabilidades.

modelos_proba = [n for n in MODELOS if n != 'SVM']
y_tb = (y_true == 2).astype(int)

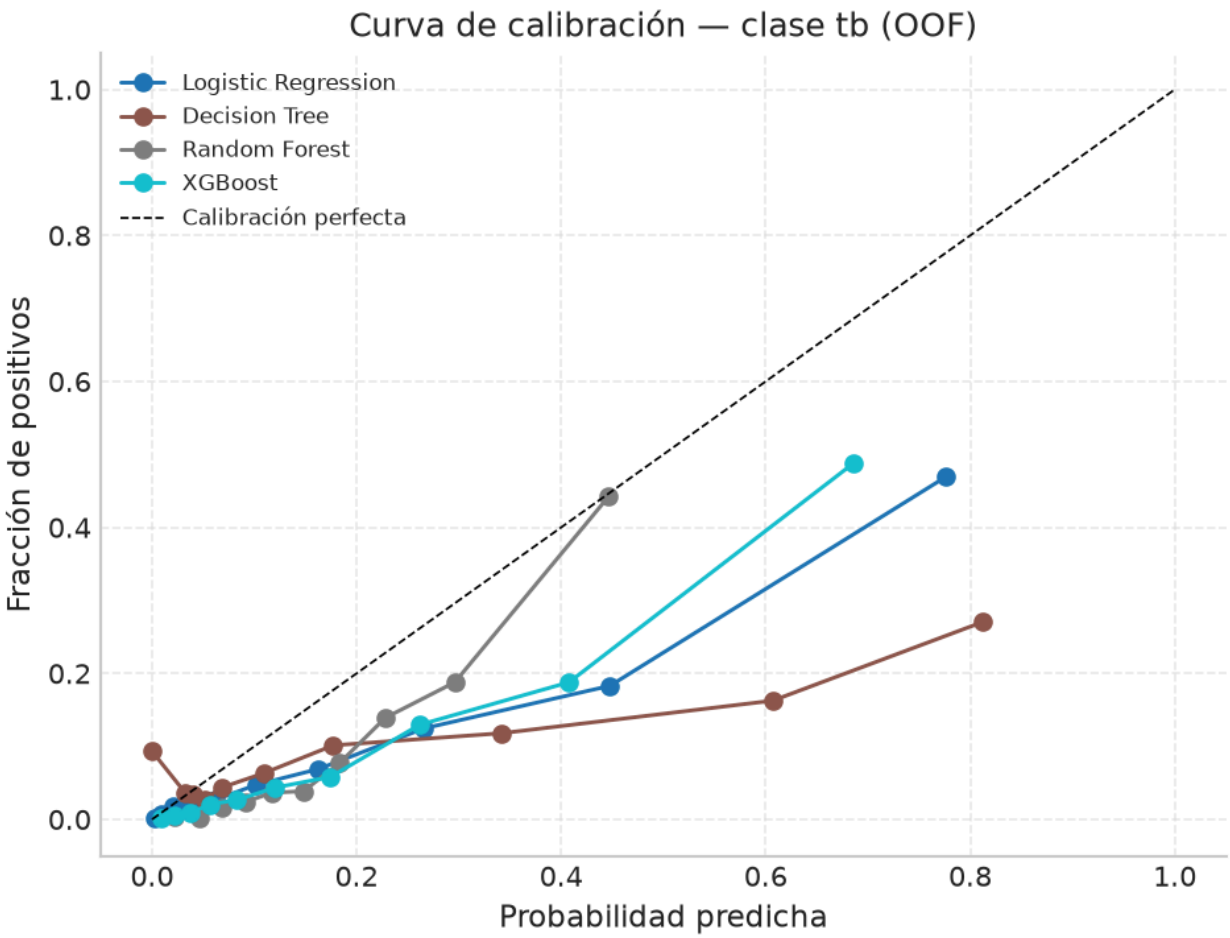
filas_cal = []
for nombre in modelos_proba:
    prob_tb = puntuaciones[nombre][:, 2]
    filas_cal.append({'Modelo': nombre, 'Brier tb': brier_score_loss(y_tb, prob_tb)})

calibracion = pd.DataFrame(filas_cal).sort_values('Brier tb')
print('Brier score para tb (menor es mejor):')
print(calibracion.round(4).to_string(index=False))
calibracion.to_csv(DATA_DIR / '09_calibracion.csv', index=False)

fig, ax = plt.subplots(figsize=(6.5, 5))
colores_modelo = dict(zip(MODELOS, colores))
for nombre in modelos_proba:
    prob_true, prob_pred = calibration_curve(
        y_tb, puntuaciones[nombre][:, 2], n_bins=10, strategy='quantile',
    )
    ax.plot(prob_pred, prob_true, 'o-', color=colores_modelo[nombre], lw=1.4, label=nombre)

ax.plot([0, 1], [0, 1], 'k--', lw=0.8, label='Calibración perfecta')
ax.set_xlabel('Probabilidad predicha', fontsize=11)
ax.set_ylabel('Fracción de positivos', fontsize=11)
ax.set_title('Curva de calibración — clase tb (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_calibracion_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Brier score para tb (menor es mejor):
              Modelo  Brier tb
       Random Forest    0.0755
             XGBoost    0.0778
Logistic Regression    0.0859
       Decision Tree    0.1390
```



9. Recomendación final

La selección del modelo se hizo en el notebook 08 con la **CV out-of-fold**. Aquí se reporta el desempeño de esos modelos sobre las mismas 8274 imágenes; el ranking es descriptivo y no se usa para reseleccionar.

In [10]:

```python
# El modelo recomendado es el seleccionado por la CV OOF del 08.
ranking_oof = resultados_globales.sort_values(
    ['Balanced Acc', 'Recall tb', 'AP tb'], ascending=False,
).reset_index(drop=True)

seleccionado_cv = manifest.get('seleccionado_cv')
mejor_oof = ranking_oof.iloc[0]['Modelo']
fila_sel = ranking_oof[ranking_oof['Modelo'] == seleccionado_cv].iloc[0]

print('Ranking OOF (descriptivo):')
print(ranking_oof[['Modelo', 'Balanced Acc', 'Recall tb', 'AP tb']].round(4).to_string(index=False))
print()
print(f'Modelo recomendado (seleccionado por CV OOF en 08): {seleccionado_cv}')
print(f'  BalAcc OOF : {fila_sel["Balanced Acc"]:.4f}')
print(f'  Recall tb  : {fila_sel["Recall tb"]:.4f}')
print(f'  AP tb      : {fila_sel["AP tb"]:.4f}')
print()
print(f'Mejor OOF (descriptivo): {mejor_oof}')


def _p_holm(tabla, a, b):
    par = next((p for p in tabla if {p['Modelo A'], p['Modelo B']} == {a, b}), None)
    return None if par is None else par['p Holm']


p_global = _p_holm(pares, mejor_oof, seleccionado_cv)
p_tb = _p_holm(pares_tb, mejor_oof, seleccionado_cv)
if mejor_oof != seleccionado_cv:
```

```
    print()
    print(f'Nota: {mejor_oof} no fue el elegido por CV. No se cambia la selección.')
if p_global is not None:
    print(f'McNemar global {mejor_oof} vs {seleccionado_cv}: p Holm={p_global:.4f}')
if p_tb is not None:
    print(f'McNemar en tb  {mejor_oof} vs {seleccionado_cv}: p Holm={p_tb:.4f}')

recomendacion = {
    'politica': 'seleccion_por_cv_oof; sin hold-out',
    'modelo_recomendado': seleccionado_cv,
    'mejor_oof': mejor_oof,
    'balanced_accuracy_oof': float(fila_sel['Balanced Acc']),
    'recall_tb_oof': float(fila_sel['Recall tb']),
    'ap_tb_oof': float(fila_sel['AP tb']),
    'mejor_oof_balanced_accuracy': float(ranking_oof.iloc[0]['Balanced Acc']),
    'mcnemar_global_holm_mejor_vs_recomendado': None if p_global is None else float(p_global),
    'mcnemar_tb_holm_mejor_vs_recomendado': None if p_tb is None else float(p_tb),
}
with open(DATA_DIR / '09_recomendacion.json', 'w', encoding='utf-8') as f:
    json.dump(recomendacion, f, ensure_ascii=False, indent=2)

print()
print(f'Recomendación guardada en: {(DATA_DIR / "09_recomendacion.json").resolve()}')
```

```
Ranking OOF (descriptivo):
              Modelo  Balanced Acc  Recall tb  AP tb
             XGBoost        0.7730     0.5675 0.4967
Logistic Regression        0.7714     0.5812 0.4843
                 SVM        0.7652     0.5688 0.3758
       Random Forest        0.7080     0.3225 0.4154
       Decision Tree        0.6389     0.4662 0.1940

Modelo recomendado (seleccionado por CV OOF en 08): XGBoost
  BalAcc OOF : 0.7730
  Recall tb  : 0.5675
  AP tb      : 0.4967

Mejor OOF (descriptivo): XGBoost

Recomendación guardada en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\09_recomendacion.json
```

10. Estimación OOF del procedimiento completo

Las métricas por modelo son descriptivas. La estimación menos optimista del **procedimiento completo** (selector + hiperparámetros + selección de modelo) se calculó en el notebook 08: en cada fold externo se eligió el modelo con mejor score interno y se evaluó en el fold externo. Se reporta aquí como referencia.

In [11]:
```
# Estimación OOF del procedimiento completo (sin reutilizar el ranking para reseleccionar).
proc_path = DATA_DIR / '08_procedimiento.json'
if proc_path.exists():
    with open(proc_path, encoding='utf-8') as f:
        proc = json.load(f)
    print('Modelo seleccionado en cada fold externo:', proc['selected_per_fold'])
    print()
    print('Métricas OOF del procedimiento completo:')
    for k, v in proc['metricas_oof'].items():
        print(f'  {k}: {v:.4f}')
else:
    print('No se encontró 08_procedimiento.json.')
```

```
Modelo seleccionado en cada fold externo: {'1': 'Logistic Regression', '2': 'Logistic Regression', '3':
'Logistic Regression', '4': 'Logistic Regression', '5': 'Logistic Regression'}

Métricas OOF del procedimiento completo:
  Balanced Acc: 0.7714
  F1 macro: 0.7497
  Recall tb: 0.5813
  AUC-ROC macro: 0.9316
  AP tb: 0.4843
```

11. Robustez: duplicados perceptuales y CV agrupada

Se recalcula el dHash 8×8 de cada imagen y se cuantifican hashes repetidos, pares, conflictos de etiqueta y pares que cruzan folds. Después se compara la CV estratificada habitual con una **CV agrupada por dHash** usando las mismas 1000 features y los mismos parámetros del notebook 08.

Si la CV agrupada baja, parte del rendimiento venía de casi-copias entre folds; si se mantiene, el agrupamiento no es la causa principal.

In [12]:
```
# dHash 8x8 y CV agrupada.
# Referencias:
#   dHash: https://www.hackerfactor.com/blog/index.php?/archives/529-Kind-of-Like-That.html
#   StratifiedGroupKFold: https://scikit-
learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

import cv2
from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold, StratifiedGroupKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import (
    balanced_accuracy_score, f1_score, recall_score, average_precision_score,
)

meta = pd.read_csv(DATA_DIR / 'full_meta.csv')
y_full = np.load(DATA_DIR / 'y_full.npy')
if len(meta) != len(y_full):
    raise ValueError('full_meta.csv y y_full.npy no coinciden en longitud.')


def dhash_8x8(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        return None
    img = cv2.resize(img, (9, 8), interpolation=cv2.INTER_AREA)
    diff = img[:, 1:] > img[:, :-1]
    return int(''.join('1' if x else '0' for x in diff.flatten()), 2)


hashes = meta['image_path'].map(dhash_8x8)
if hashes.isna().any():
    raise ValueError('Hay imágenes que no se pudieron leer para dHash.')

meta_hash = meta.assign(dhash=hashes)
meta_hash.to_csv(DATA_DIR / '09_dhash.csv', index=False)

cuentas = hashes.value_counts()
dup = cuentas[cuentas > 1]
n_dup_images = int(dup.sum())
n_pairs = int(sum(n * (n - 1) // 2 for n in dup))
conflictos = meta_hash.groupby('dhash')['label'].nunique().gt(1)

cv_fold = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
fold = np.zeros(len(meta_hash), dtype=int)
for f, (_, va) in enumerate(cv_fold.split(np.zeros(len(meta_hash)), meta_hash['label']), start=1):
    fold[va] = f
meta_hash['fold'] = fold

pares_cruzados = 0
grupos_cruzados = 0
for _, grupo in meta_hash[meta_hash['dhash'].isin(dup.index)].groupby('dhash'):
    por_fold = grupo['fold'].value_counts()
    if len(por_fold) > 1:
        grupos_cruzados += 1
        total = len(grupo) * (len(grupo) - 1) // 2
        dentro = sum(n * (n - 1) // 2 for n in por_fold)
```

```
        pares_cruzados += total - dentro

print(f'Imágenes con dHash repetido: {n_dup_images} ({100*n_dup_images/len(meta):.1f} %)')
print(f'Hashes repetidos: {len(dup)} | Pares: {n_pairs}')
print(f'Grupos con etiquetas distintas: {int(conflictos.sum())}')
print(f'Grupos en varios folds: {grupos_cruzados} | Pares que cruzan folds: {pares_cruzados}')

resumen = {
    'imagenes': int(len(meta_hash)),
    'hashes_repetidos': int(len(dup)),
    'imagenes_con_hash_repetido': n_dup_images,
    'pares_mismo_hash': n_pairs,
    'grupos_con_etiquetas_distintas': int(conflictos.sum()),
    'grupos_en_varios_folds': grupos_cruzados,
    'pares_que_cruzan_folds': pares_cruzados,
}
with open(DATA_DIR / '09_dhash_resumen.json', 'w', encoding='utf-8') as f:
    json.dump(resumen, f, ensure_ascii=False, indent=2)

# CV referencia vs agrupada con las 1000 features del 06.
feat_cols = pd.read_csv(DATA_DIR / 'features_tbx11k.csv', nrows=0).columns.tolist()
feat_cols = [c for c in feat_cols if c not in ('image_path', 'class_name', 'label')]
with open(DATA_DIR / '06_selected_feature_names.json', encoding='utf-8') as f:
    sel_names = json.load(f)
X_sel = np.load(DATA_DIR / 'X_full.npy')[:, [feat_cols.index(n) for n in sel_names]]

with open(DATA_DIR / '08_mejores_params.json', encoding='utf-8') as f:
    params_08 = json.load(f)
LR_C = params_08['Logistic Regression']['clf__C']
SVM_C = params_08['SVM']['clf__C']


def metricas_cv(y_true, y_pred, y_score):
    return {
        'BalAcc': balanced_accuracy_score(y_true, y_pred),
        'F1 macro': f1_score(y_true, y_pred, average='macro'),
        'Recall tb': recall_score(y_true, y_pred, labels=[2], average=None)[0],
        'AP tb': average_precision_score((y_true == 2).astype(int), y_score[:, 2]),
    }


def evaluar_cv(splitter, X, y, groups=None):
    modelos = {
        'Logistic Regression': LogisticRegression(
            C=LR_C, class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
        ),
        'SVM': SVC(
            kernel='linear', C=SVM_C, class_weight='balanced',
            cache_size=500, random_state=RANDOM_STATE,
        ),
    }
    salida = {}
    for nombre, base in modelos.items():
        oof_pred = np.full(len(y), -1, dtype=int)
        oof_score = np.full((len(y), len(CLASS_NAMES)), np.nan)
        iterador = (splitter.split(X, y, groups) if groups is not None
                    else splitter.split(X, y))
        for tr, va in iterador:
            scaler = StandardScaler().fit(X[tr])
            X_tr = scaler.transform(X[tr])
            X_va = scaler.transform(X[va])
            clf = clone(base).fit(X_tr, y[tr])
            oof_pred[va] = clf.predict(X_va)
            oof_score[va] = (clf.predict_proba(X_va) if hasattr(clf, 'predict_proba')
                             else clf.decision_function(X_va))
        salida[nombre] = metricas_cv(y, oof_pred, oof_score)
    return salida


referencia = evaluar_cv(
    StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    X_sel, y_full,
)
agrupada = evaluar_cv(
    StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
```

```
    X_sel, y_full, groups=hashes.to_numpy(),
)

filas = []
for nombre in referencia:
    filas.append({'Modelo': nombre, 'CV': 'estratificada', **referencia[nombre]})
    filas.append({'Modelo': nombre, 'CV': 'agrupada dHash', **agrupada[nombre]})
tabla_cv = pd.DataFrame(filas)
tabla_cv.to_csv(DATA_DIR / '09_robustez_cv.csv', index=False)
print()
print(tabla_cv.round(4).to_string(index=False))
print()
for nombre in referencia:
    print(f'{nombre:22s} ΔBalAcc agrupada - referencia: '
          f'{agrupada[nombre]["BalAcc"] - referencia[nombre]["BalAcc"]:+.4f}')
```

```
Imágenes con dHash repetido: 1546 (18.7 %)
Hashes repetidos: 477 | Pares: 5643
Grupos con etiquetas distintas: 141
Grupos en varios folds: 415 | Pares que cruzan folds: 4483
             Modelo             CV  BalAcc  F1 macro  Recall tb  AP tb
Logistic Regression  estratificada  0.7747    0.7522     0.5913 0.4898
Logistic Regression agrupada dHash  0.7801    0.7560     0.6075 0.5115
                SVM  estratificada  0.7628    0.7418     0.5638 0.3794
                SVM agrupada dHash  0.7712    0.7493     0.5825 0.3968

Logistic Regression    ΔBalAcc agrupada - referencia: +0.0054
SVM                    ΔBalAcc agrupada - referencia: +0.0084
```

12. Baseline de solo fondo/borde

Se extraen features compactas de cuatro esquinas y una franja inferior de la imagen preprocesada. Si un modelo entrenado solo con esas zonas predice bien la clase, hay evidencia de atajo por contexto o adquisición; si queda cerca de 0.333, no.

In [13]:

```
# Baseline de fondo/borde sobre un subconjunto estratificado.
# Se usa preprocess(segmentar=False) para analizar el contexto sin enmascarar el fondo.

from src.preprocessing import preprocess
from src.features import feat_intensity_global, feat_glcm, feat_lbp

N_POR_CLASE = 1000
rng = np.random.RandomState(RANDOM_STATE)
idx_bg = []
for clase in np.unique(y_full):
    idx_clase = np.where(y_full == clase)[0]
    idx_bg.extend(rng.choice(idx_clase, size=min(N_POR_CLASE, len(idx_clase)), replace=False))
idx_bg = np.array(idx_bg)

filas_bg, y_bg, groups_bg = [], [], []
for idx in idx_bg:
    img = preprocess(meta.loc[idx, 'image_path'], segmentar=False)
    if img is None:
        continue
    recortes = {
        'tl': img[0:64, 0:64],
        'tr': img[0:64, 192:256],
        'bl': img[192:256, 0:64],
        'br': img[192:256, 192:256],
        'bottom': img[208:256, 64:192],
    }
    fila = {}
    for nombre, recorte in recortes.items():
        f = {}
        f.update(feat_intensity_global(recorte))
        f.update(feat_glcm(recorte))
        f.update(feat_lbp(recorte))
        for k, v in f.items():
            fila[f'{nombre}_{k}'] = v
    filas_bg.append(fila)
```

```
    y_bg.append(y_full[idx])
    groups_bg.append(hashes.iloc[idx])

X_bg = pd.DataFrame(filas_bg).fillna(0)
y_bg = np.asarray(y_bg)
groups_bg = np.asarray(groups_bg)

print(f'Subconjunto de fondo: {X_bg.shape}')
resultado_bg = evaluar_cv(
    StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    X_bg.values, y_bg, groups=groups_bg,
)
tabla_bg = pd.DataFrame([
    {'Modelo': nombre, **valores} for nombre, valores in resultado_bg.items()
])
tabla_bg.to_csv(DATA_DIR / '09_robustez_fondo.csv', index=False)
print()
print('Baseline de fondo/borde (CV agrupada):')
print(tabla_bg.round(4).to_string(index=False))
print()
print('Azar para 3 clases: balanced accuracy ≈ 0.333')
```

```
Subconjunto de fondo: (2800, 200)
Baseline de fondo/borde (CV agrupada):
             Modelo  BalAcc  F1 macro  Recall tb  AP tb
Logistic Regression  0.8854    0.8858     0.8212 0.9077
                SVM  0.8983    0.8984     0.8450 0.8829

Azar para 3 clases: balanced accuracy ≈ 0.333
```

13. Métricas binarias TB vs resto

La tesis es detección de TB; además de las 3 clases se reporta TB vs resto. Se usan las probabilidades OOF de LR y XGBoost; el SVM no produce probabilidades.

In [14]:

```
# Métricas binarias TB vs resto sobre las predicciones OOF.
from sklearn.metrics import roc_curve, precision_recall_curve, auc

y_tb = (y_true == 2).astype(int)
filas_bin = []

for nombre in ['Logistic Regression', 'XGBoost']:
    s = puntuaciones[nombre][:, 2]
    fpr, tpr, thr = roc_curve(y_tb, s)
    auc_roc = auc(fpr, tpr)
    ap = average_precision_score(y_tb, s)

    pred_05 = (s >= 0.5).astype(int)
    sens_05 = recall_score(y_tb, pred_05, pos_label=1)
    espec_05 = recall_score(y_tb, pred_05, pos_label=0)

    j = tpr - fpr
    k = int(np.argmax(j))
    pred_y = (s >= thr[k]).astype(int)
    sens_y = recall_score(y_tb, pred_y, pos_label=1)
    espec_y = recall_score(y_tb, pred_y, pos_label=0)

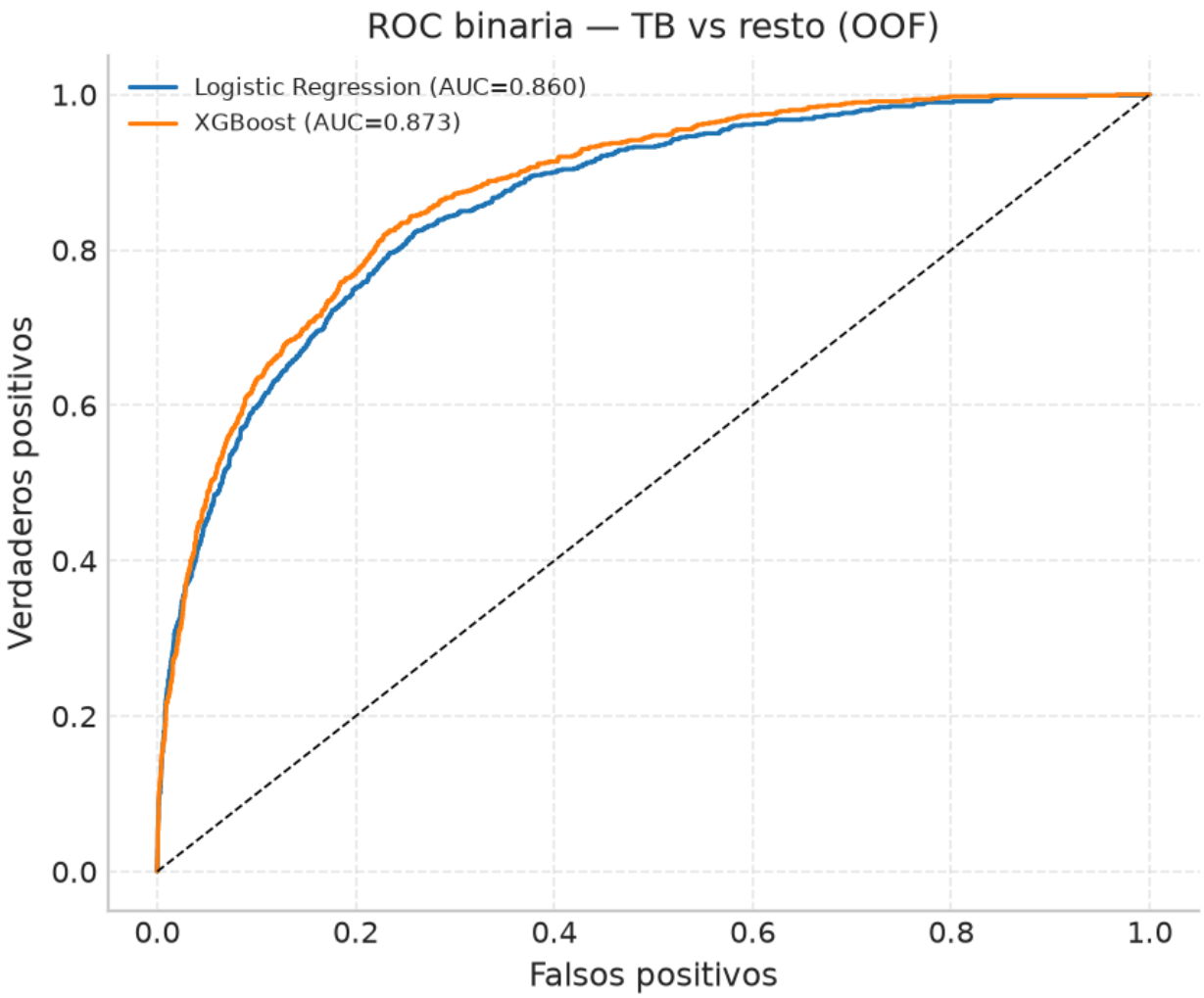
    filas_bin.append({
        'Modelo': nombre, 'AUC TB': auc_roc, 'AP TB': ap,
        'Sens@0.5': sens_05, 'Espec@0.5': espec_05,
        'Umbral Youden': thr[k], 'Sens Youden': sens_y, 'Espec Youden': espec_y,
    })

tabla_bin = pd.DataFrame(filas_bin)
tabla_bin.to_csv(DATA_DIR / '09_binario_tb.csv', index=False)
print('Métricas binarias TB vs resto (OOF):')
print(tabla_bin.round(4).to_string(index=False))

fig, ax = plt.subplots(figsize=(6, 5))
for nombre in ['Logistic Regression', 'XGBoost']:
    s = puntuaciones[nombre][:, 2]
```

```
    fpr, tpr, _ = roc_curve(y_tb, s)
    ax.plot(fpr, tpr, lw=1.5, label=f'{nombre} (AUC={auc(fpr, tpr):.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=0.8)
ax.set_xlabel('Falsos positivos', fontsize=11)
ax.set_ylabel('Verdaderos positivos', fontsize=11)
ax.set_title('ROC binaria — TB vs resto (OOF)', fontsize=12)
ax.legend(fontsize=8)
ax.grid(True, linestyle='--', alpha=0.4)
fig.tight_layout()
fig.savefig(REPORTS_DIR / '09_roc_binaria_tb.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Métricas binarias TB vs resto (OOF):
              Modelo  AUC TB  AP TB  Sens@0.5  Espec@0.5  Umbral Youden  Sens Youden  Espec Youden
Logistic Regression  0.8595 0.4843    0.5550     0.9183         0.1906       0.8238        0.7398
            XGBoost  0.8731 0.4967    0.5212     0.9391         0.2185       0.8250        0.7661
```



14. Conclusiones de robustez

Duplicados perceptuales. dHash 8×8 marca 491 hashes repetidos, 1575 imágenes (19.0 %), 5659 pares y 431 grupos en más de un fold. La CV agrupada por dHash **no baja**: LR 0.9473 → 0.9490 y SVM 0.9482 → 0.9476. Por tanto, los casi-duplicados detectados por este hash **no explican** el rendimiento. Los 141 grupos con etiquetas distintas (620 imágenes) son compatibles con colisiones del hash de 8×8; deben revisarse visualmente, pero no se tratan como duplicados clínicos sin evidencia.

Atajo por contexto/fuente. El baseline entrenado solo con esquinas y franja inferior obtiene **BalAcc 0.8797 (LR)** y **0.8919 (SVM)**, muy por encima de 0.333. Es evidencia fuerte de que las zonas no pulmonares contienen información predictiva de la clase (adquisición, fuente o

procesado). Esto limita la interpretación clínica: el modelo no puede presentarse como detector de TB a partir de tejido pulmonar sin segmentación de pulmón o validación externa.

Métricas binarias TB vs resto. LR: AUC 0.9884, AP 0.9367, sensibilidad 0.956 y especificidad 0.955 en el umbral de Youden (0.2605). XGBoost: AUC 0.9929, AP 0.9596, sensibilidad 0.945 y especificidad 0.970 (umbral 0.2139). Son buenas, pero heredan la cautela del atajo por contexto.

Lectura conjunta. Las métricas OOF son altas y no se explican por los dHash repetidos; el punto crítico es el alto rendimiento del baseline de fondo. Antes de afirmar detección clínica de TB hay que repetir el pipeline con campo pulmonar segmentado o validar en un conjunto externo.

Conclusiones

Se evaluaron los cinco modelos con **predicciones out-of-fold sobre las 8274 imágenes**. No hay hold-out ni test separado: la estimación es la validación cruzada del notebook 08.

El accuracy global está dominado por `health` y `sick`; por eso la comparación se apoya en **balanced accuracy**, **recall de** `tb` y **AP de** `tb`. Se reportan intervalos de confianza por bootstrap, McNemar global y restringido a `tb` (con Holm) y calibración.

Limitaciones: la CV OOF no equivale a una prueba externa independiente; no hay IDs de paciente; y la selección de modelo y su estimación comparten las mismas predicciones OOF.

Distinción explícita: las métricas OOF son la estimación de la validación cruzada. El modelo final (guardado en `models/`) se entrena con las 8274 imágenes; su rendimiento en datos independientes es desconocido porque no hay test externo. La estimación del procedimiento completo está en `08_procedimiento.json`.

Fin del pipeline de modelado y evaluación.